

Окончательная архитектура проекта

Основные источники данных

ARB

Содержит локализованные данные.

Пример:

- строки интерфейса;
- метаописания;
- локализованные тексты.

ARB является одним из источников данных и не содержит никакой логики преобразования.

config.json

Содержит глобальные настройки проекта.

Например:

- базовый URL сайта;
- название приложения;
- идентификаторы;
- пути;
- параметры генерации.

Конфигурация одинакова для всех локалей и не зависит от ARB.

schema.json

Является единственным местом, где описывается структура генерации.

Схема декларативно определяет:

- из какого источника брать данные;
- какие проверки необходимо выполнить;
- куда записывать результат;
- какие значения являются обязательными;
- какие типы ожидаются.

Схема не содержит сценариев выполнения, последовательностей операций или пользовательского кода.

Основные компоненты

SourceResolver

Отвечает исключительно за получение значения из указанного источника.

Например:

- ARB;
- config.json;
- встроенные системные значения;
- дополнительные источники в будущем.

SourceResolver ничего не знает о HTML, JSON-LD или правилах валидации.

Validator

Выполняет проверки, если они описаны в schema.json.

Например:

- проверка типа;
- обязательность поля;
- минимальная длина строки;
- регулярные выражения;
- допустимые значения.

Если для поля проверки отсутствуют, Validator просто пропускает этот этап.

Target

Отвечает исключительно за запись значения в конечное представление.

Например:

- HTML;
- JSON-LD;
- Open Graph;
- Meta-теги;
- любые новые форматы.

Каждый Target знает только особенности своего формата и ничего не знает об источниках данных.

HtmlDocument

Единственное место в проекте, которое работает с HTML.

Именно этот класс:

- открывает HTML-документ;
- ищет необходимые элементы;
- изменяет атрибуты;
- изменяет содержимое тегов;
- сохраняет результат.

Остальная часть проекта не взаимодействует с HTML напрямую.

Принципы архитектуры

- Вся структура генерации описывается исключительно в `schema.json`.
 - Логика получения данных отделена от логики записи.
 - Проверки выполняются только при наличии соответствующего описания в схеме.
 - Каждый класс имеет одну ответственность.
 - Добавление нового источника данных не требует изменения `Target`.
 - Добавление нового `Target` не требует изменения `SourceResolver`.
 - Работа с HTML полностью изолирована внутри `HtmlDocument`.
 - Архитектура остается декларативной и не использует пользовательский язык преобразований.
-

Итог

Архитектура одновременно остается гибкой и не перегруженной.

Ответственность за структуру данных полностью сосредоточена в `schema.json`, а вся техническая сложность распределена между небольшими специализированными классами Dart.

Такой подход делает проект легче тестировать, сопровождать и расширять без изменения уже существующих компонентов.